

\bg = #000000

\fg = #ffffff

\highlight = #d03189

\lowlight = #aaaaaa

\code = #ddd dbb

\bullet = "â€¢"

The right kind of abstraction
John Cinnamond

GopherCon UK 2025

Go's philosophy emphasizes avoiding unnecessary abstractions

Go's philosophy emphasizes avoiding abstractions

What does
unnecessary
mean here?

I'm going to use unnecessary abstractions

What kind of abstractions are unnecessary?

Java Spring
ORMs

What about map/reduce?

What about map/reduce?

```
names := []string{}  
for _, u := range users {  
    names = append(names, u.Name)  
}
```

```
ids := []uuid.UUID{}  
for _, u := range users {  
    ids = append(ids, u.ID)  
}
```

names

names

names

ids

ids

ids

string

u.Name

uuid.UUID

u.ID

What about map/reduce?

```
func extract[A any] (fn func(User) A, users []User) []A {  
  fn func(User) A  
  A  
  
  out := []A{}  
  for _, u := range users {  
    out = append(out, fn(u))  
  }  
  return out  
}  
A  
fn(u)  
ids := extract(func(u User) string { u.ID }, users)
```

What kind of abstractions are unnecessary?

map/reduce

generics

interfaces?


```
func parseBody(r io.Reader) {  
    bytes, err := io.ReadAll(r)  
    // ...  
}
```

```
func parseFileBody(f os.File) {  
    var bytes := make([]byte, 1024)  
    n, err := f.Read(&b)  
    // ...  
}
```

```
func parseBytesBody(b []byte) {  
    // ...  
}
```

What kind of abstractions are unnecessary?

interfaces

functions?

assembler?

machine code?

Programming without abstractions
is just screaming electrical impluses
into silicon wafers

This is obviously absurd

Go's philosophy emphasizes avoiding unnecessary abstractions

unnecessary != technically possible to program without them

Go's philosophy emphasizes avoiding
unnecessary abstractions
unnecessary abstractions
inappropriate abstractions

so...

what is an
inappropriate
abstraction?

it depends who you ask

Appropriate abstractions are ones that I write.
It's other people's abstractions that are the problem.

The
appropriateness
of abstractions

is subjective

and contextual

But it's not arbitrary

It's not like asking "what is your favourite color?"

Maybe I can't define a bad abstraction...

But I know it when I see it

In general, there are a bunch of abstractions
that we can all agree are "bad"

...and a bunch of abstractions
that we can all agree are "good"

...and an awkward bunch in the middle
where we don't agree

Which of these awkward abstractions are appropriate?

I can't answer that.
(soz)

but all is not lost!

"\"good\" and \"bad\" have meaning"

when talking about abstractions

"... it's just a bit hard to explain"

This talk is about learning how
to articulate that meaning

Part 1

What is an abstraction, and why would I want one?

An abstraction
is a representation

that deliberately removes some detail
of the thing being represented

An abstraction is a representation

It's not a thing in itself

It's always an abstraction of...

```
func (f nameFilter) Check(p Product) bool {  
    return p.name == f.name  
}
```

```
func (f priceFilter) Check(p Product) bool {  
    return p.price <= f.maxPrice  
}
```

```
func (stockFilter) Check(p Product) bool {  
    return p.hasStock  
}
```

```
type Filter interface {  
    Check(Product) bool  
}
```

Abstractions are usually
indefinite
references.

(they can refer to many different things)

Abstractions are usually
open
references.

(the set of things they refer to can
change over time)

that deliberately removes some detail

There are many kinds of references that try to be as accurate as possible.

Abstractions are different.

They only want to represent some part of the subject.

A model car is a representation

[image: car.jpg]

That tries to be as accurate as possible

[image: gopher.jpg]

The go gopher
is a representation
that deliberately
omits details

(and lifts heavy)

An abstraction
is a representation
that deliberately removes some detail
of the thing being represented

But why would we want to do that?

- to avoid repeated code
- to use different structures in the same way
- to defer (thinking about) some details
- to represent domain concepts
- to reveal something about the implementation

Ultimately, we want to use abstractions
because they enable us to solve
complicated problems

Not all abstractions are the same

Some abstractions are based on
how something works

You see some repeated code

```
if user.isAdmin {  
  deleteTheThing()  
}
```

```
if user.isAdmin {  
  addPermission()  
}
```

Not all abstractions are the same

Some abstractions are based on
how something works

You create an abstraction

```
func AdminDoer(u User, fn func ()) {  
  if u.isAdmin {  
    fn()  
  }  
}
```

(I apologise for the quality of that abstraction)

Not all abstractions are the same

Some abstractions are based on
how something works

Some abstractions are based on
what something does

```
type Reader interface {  
    Read(p []byte) (n int, err error)  
}
```

```
type Stringer interface {  
    String() string  
}
```

Not all abstractions are the same

Some abstractions are based on
how something works

Some abstractions are based on
what something does

Some abstractions are based on
what something is

```
type User struct {  
    ID      uuid.UUID  
    Name    string  
    Email   string  
    CreatedAt time.Time  
}
```

Not all abstractions are the same

Some abstractions are based on
how something works

Some abstractions are based on
what something does

Some abstractions are based on
what something is

Most abstractions are some mixture of these

Part 2

How to think about abstractions

Part 2a

Software development

is a social activity

A software development team

is a social group

Societies develop customs and lore

Fitting into society means

adopting these customs

Over time, societies become conservative

The customs and beliefs ossify

Society can be changed

but it changes slowly, and reluctantly

You can't be a member of a society

without working with its customs and beliefs

To introduce a new abstraction you need to think about

how well does it fit with existing lore?

does it match the established customs and beliefs?

how much work is it to change the society?

Do the benefits of the new abstraction

outweight the costs of introducing it?

There is a social cost to introducing
a new abstraction

So make sure it's worth it

Part 2b

ready-to-hand

vs

present-at-hand

[image: being_and_time.jpg]

Martin Heidegger

Sein und Zeit (Being and Time)

1927

We normally encounter objects as
ready-to-hand

We consider them in terms of trying to achieve
some other task

We don't normally consider objects
independently
theoretically

Consider a hammer

[image: hammer.jpg]

We normally consider a hammer
in terms of hammering something

We don't think about how it
is constructed

We don't think about what it
means to be a hammer

" "

" "

" "

" "

" "

" "

If it breaks
we forget about hammering.

We become aware of
the hammer itself.

Consider a for loop

```
for _, u := range users {  
    // ...  
}
```

We normally consider it in terms of
doing something repeatedly

We don't think about how it is implemented

We don't think about
what a for loop is

A
for loop
is ready-to-hand

What about generics?

generics?

patterns?

monads?

It depends how familiar we are with them

Unfamiliar abstractions are

present-at-hand

Broken abstractions are

present-at-hand

"... and "

present-at-hand

abstractions

are bad abstractions

They distract us from what we're trying to do.

Unfamiliar abstractions can become

familiar

through education and practice

present-at-hand
abstractions

present-at-hand

can become
ready-to-hand

(just don't forget to take your team with you)

Part 2c

Essential Truth

[image: kant.jpg]

Immanuel Kant

Critique of Pure Reason

1781

Analytic vs Synthetic propositions

The truth of an
analytic
proposition

is derived from its content

e.g.

triangles have three sides

all bachelors are unmarried

unnecessary abstractions are unnecessary

Analytic vs Synthetic propositions

The truth of a
synthetic
proposition

is derived from
outside
of its content

e.g.

it is raining
the book is on the table

Analytic
propositions which are true

are
necessarily
true.

Learning new facts cannot make them false.

Synthetic
propositions which are true

are
circumstantially
true.

They can become false if circumstances change.

There is a certain appeal to
analytic
truth

Because it is
necessarily
true

[image: putnam.jpg]

Hilary Putnam

The Analytic and the Synthetic
Philosophical Papers, Volume 2
1979

it is far better to proceed on the idea that
statements fall into three kinds - analytic,
synthetic, and lots-of-other-things

There are propositions that are true by definition
(
necessary
truths)

unnecessary abstractions are unnecessary

There are propositions that are circumstantially true
(
coincidental
truths)
I am giving a talk

There are propositions that are definitely true,
but not by definition
(
essential
truths)
All matter is made from atoms

Essential truths get overturned

as we learn more about the world

definitely <-- --- --- --- --- --- --- --> maybe
true true

analytic synthetic

essential coincidental
truth truth

established new ideas
theory

Abstractions have a truthfulness

Is an abstraction a
true
representation

of an idea, or of some implementation?

Does an abstraction always represent
every instance of an idea or implementation?

```
res := []Product{}
for _, p := range products {
    if p.price < maxPrice {
        res = append(res, p)
    }
}
```

```
res := []Product{}
for _, p := range products {
    if p.inStock {
        res = append(res, p)
    }
}
```

```
type Filter = func(Product) bool
```

```
func apply(products [], fs ...Filter) []products {  
    // apply all filters to the list of products  
}
```

`type Filter = func(Product) bool`

Is this abstraction
essentially
true?

Does it represent all possible filters?

That depends on
the way the world is
reality

type Filter = func(Product) bool

Is this abstraction
essentially
true?

price < maxPrice
inStock
size == medium
onSale

All good

type Filter = func(Product) bool

Is this abstraction
essentially
true?

first product with any given name

type Filter = func(Product,
[]Product, int) bool
map[string]struct{}) bool

Not good

Good abstractions are

essentially true

Questionable abstractions are

coincidentally true

Essential truths can be overturned as we learn more about reality

Part 2d

Algebra

Algebra is a branch of mathematics
that deals with abstract systems

Algebra has been very influential in computer science and, to a lesser extent, in programming.

From group theory

e.g.,
monoids, semi-rings, lattices

From category theory

e.g.,
functors, monads, arrows

From type theory

e.g.,
dependent types

I don't want to talk about particular algebras.

I want to talk about an aspect of how algebra works

"... or rather, algebra doesn't "
work

algebra
is

You don't
do
an algebra
You
recognise
it

```
type Filter = func(Product) bool
```

```
func join(f1, f2 Filter) Filter {  
  return func(p Product) {  
    return f1(p) && f2(p)  
  }  
}
```

This is a monoid.

We haven't
used
a monoid.

It just
is
one.

A
monoid
consists of:

A set S

A binary operation $S \times S \rightarrow S$, denoted by $*$

satisfying two axioms

Associativity: $\forall s, t, u \in S, (s * t) * u = s * (t * u)$

Identity: $\exists e \in S, \forall s \in S, s * e = s = e * s$

ain't nobody got time for that

It doesn't matter whether or not you know what a monoid is.

That doesn't change the fact that this is a monoid

```
type Filter = func(Product) bool
func join(f1, f2 Filter) Filter {
  return func(p Product) {
    return f1(p) && f2(p)
  }
}
```


This is a monoid

This isn't a
judgement

We haven't done any
work

to make it a monoid.

It's just reality

A lot of abstractions are like this in programming

We don't
create
them.

We
recognise
them.

Their value comes in how they reveal truths
about our code
...and in how they allow us to use different
structures in the same way

e.g,
because I know this is a monoid...

```
type Filter = func(Product) bool
func join(f1, f2 Filter) Filter {
  return func(p Product) {
    return f1(p) && f2(p)
  }
}
```

I know I can collapse a list of
filters into a single filter
That's just a consequence of it
being a monoid

Programming with abstractions
is about understanding your code
and recognising patterns within it.

Good
abstractions come from

recognising patterns that are

useful
in some way.

Part 3

The right kind of abstraction

Which abstractions are appropriate?

I still can't give you an answer to this.
(soz)

Do the benefits of the new abstraction
outweigh the costs of introducing it?

It's always going to be subjective.
It's always going to be contextual.

But I can tell you how to think about this...

Be suspicious of abstractions

Don't introduce an abstraction

just because it seems neat

Avoid premature abstraction

There are lots of bad abstractions out there

The wrong abstraction makes your code

harder to
understand
maintain
extend

prefer duplication over the wrong abstraction

<https://sandimetz.com/blog/2016/1/20/the-wrong-abstraction>

however

The right abstractions supercharge

our ability to write code

So... you think you've found a good abstraction

What now?

1.

Understand the benefit

What problem are you trying to solve?

avoiding repetition?

revealing new ways to structure code?

deferring details?

identifying a domain concept?

describing some truth about the implementation?

If you don't understand the benefit...

then just stop

2.

Think about the social cost

Does it fit with the culture and beliefs of the team?

Is it the kind of abstraction they already use?

"... and if not ..."

Is it worth the cost of challenging the social norm?

3.

Will the abstraction be a distraction?

Is the abstraction
ready-to-hand

do you use it without thinking?

Or is it
present-at-hand

are you distracted by the abstraction?

Avoid present-at-hand abstractions

4.

Is the abstraction
essential
"?"

Is it a
true
representation of the code

you're trying to abstract away from?

How well do you know the code you're abstracting?

How well do you understand the abstraction?

Are new examples likely to emerge?

Learn the reality

Respect the reality

5.

Have you
recognised
some

fundamental pattern in the code?

Are you exploiting a pattern that's really there?

Or are you
doing work

to force an abstraction onto the code?

Good abstractions
emerge

Good abstractions

1. have a clear benefit
2. fit well with the team culture
3. are ready-to-hand
(can be used without thinking about them)
4. are essential representations
(always represent all examples)
5. are emergent
(are a recognition of patterns that already exist)

this is a lot

Good abstractions are hard to find

Do you really think you've found a good abstraction?

Are you really that good?

Yes!

(maybe)

I believe in you!

Finding good abstractions requires a lot of work
but you can do it.

...and don't be put off!

The payoff from finding a good abstraction
can be huge.

Remember that code is cheap!

think you've found a good abstraction?

Just try it.

See what happens.

The important thing...
is to think about what you're doing.
and to check in with reality.

Stay skeptical and curious

Spend the time and effort required

Then you can find the right kind of abstraction

and write great code

Thank you!

The right kind of abstraction

John Cinnamond

GopherCon UK 2025